

DAA - Furrer - Guggisberg - Troeltsch

Introduction Kotlin

val immutable - assignable une fois	var mutable
val p : Person?; p?.name	Null safety: Pour chaque type T, il existe T? qui prend T ou null
c?.name ?: "default"	Elvis Operator: Prendre valeur en cas de null
fun String.doSomething():String {...}	Extension functions: ajouter fonctions à classes sans sources
class Person(var i:Int) {init {...}}	init: constructeur par défaut
constructor(i:Int,j:Int):this(i) {...}	constructor: constructeurs supplémentaire
fun add(x: Int, y: Int):Int {...}	Arguments nommés possible: add(y = 1, x = 2)
class Student(...): Person(...) {...}	Héritage: appel super-constructeur. open class Person() {...}
interface Flying { val w:Wings fun fly(){...} }	class Bird: Flying { override val wings:Wings=... }
data class Person(var name:String)	Génère: getter - setters - toString()
val(name, surname) = p	Déstructuration pour data class
var p2 = p1.copy(surname="Bob")	Copie pour data class, arguments optionnels
val s = "Person[name:\${p.name},id:\$id"]	String templates
Collections	List, MutableList, Set, MutableSet, Map, MutableMap
val l = listOf(1, 2, 3, 4)	l.any {...} l.count {...} l.max() l.filter {...} l.map {...} l.partition {...} l+l etc
Operators overload:	Array: get(i) set(i) a..b b.rangeTo(b) a in b b.contains(a)
Unary: unaryPlus() not() inc() dec()	Binary: plus(b) minus(b) times(b) div(b) mod(b) plusAssign()
val res = when(x) {1->"a" else->{"b"}}	When - Pattern Matching
p?.let {it.name="Bob" println(it)}	Scope functions - let - retourne dernière ligne du bloc
val p2 = p.apply {this.name="Bob" println(this)}	Scope functions - apply - retourne l'objet lui-même
BufferedWriter(FileWriter("a.txt")).use{ it.appendLine("Hello") }	Scope functions - use - appelé sur objets <i>Closeable</i> , ferme la variable it automatiquement en fin de bloc ou en cas d'exception
companion object {var a fun getA():A}	Companion objects contient variables/fonctions statiques

Ressources

Manifest	décrit informations essentielles pour build, OS, et store
Doit contenir	Composants d'app(Activités, Services, Broadcast receivers, Content providers), Permissions, Fonctionnalités (hard/software)
Ressources	différentes ressources et classe <i>R</i>
Contextualisation - 🚩 Lire doc pour ordre	<i>fr sw600dp night land</i> densité écran: <i>hdpi</i> taille écran: <i>normal</i>
Valeurs - string.xml	peut être regroupées en tableau
Placeholders	<string name="welcome">Hello, %1\$s!</string>

Plurals - zero one two few many	<pre><plurals name="test"> <item quantity="one">one</item> <item quantity="other">more</item> </plurals></pre>
Dimensions - dimension.xml	dp: density independent pixel, sp: scale independent pixel
Couleurs - colors.xml	-> themes.xml
Drawables	
Bitmap - png, webp, jpeg, gif	Plusieurs résolutions pour chaque type d'écran
Vector - xml	outil pour convertir depuis svg ou psd
Nine-Patch - *.9.[png]	Redimensionnement contrôlé
State List	Drawable matchant différents states de la vue
Level List	Drawable matchant une valeur numérique
Layouts - LinearLayout - RelativeLayout - ConstraintLayout - ScrollView	possible de les imbriquer mais moins bonnes performances. ScrollView est vertical (mais HorizontalScrollView existe)
classe R	regroupe les ids des ressources. Accessible code ou ressources
Build - Gradle	build.gradle app et projet
Dépendances - Maven	via <i>build.gradle.kts</i> ou <i>libs.versions.toml</i> (mieux)

Layout

Activity - Stack	Doivent être déclarées dans le manifest
Cycle de vie - Inact->Stopped->Paused->Active->Paused->Stopped->Inact	Active:foreground, can not be killed Paused:visible, no focus Stopped:invisible Inactive:temporary when created/killed
État activité	activités sur stack peuvent être détruites. changement de config va recréer activité active
Sauvegarde-Restoration	onSaveInstanceState -> Bundle -> onCreate/ onRestoreInstanceState
Intents	Lance une activité sur la stack avec paramètres éventuels
Intents Explicites - activité précise	Intents Implicites - appel générique via un lien/type intent
Contracts	résultat d'une activité (moderne, non déprécié)

Fragment

Activités, Fragments, Services
Interface graphique
LiveData et MVVM
Persistence des données